6. 증명 생성 프로세스 상세 분석

개요

Stwo의 증명 생성 프로세스는 Circle STARK 프로토콜의 핵심으로, 계산 제약 조건들을 수학적으로 검증 가능한 형태로 변환하는 복잡한 과정입니다. 이 프로세스는 크게 두 단계로 나뉩니다: 컴포지션 다항식(Composition Polynomial) 생성과 OODS(Out-Of-Domain Sampling) 구현입니다.

6.1 컴포지션 다항식 생성 과정 분석

6.1.1 컴포지션 다항식의 개념과 역할

컴포지션 다항식은 여러 개의 제약 조건 다항식들을 하나의 다항식으로 결합하는 핵심 구조입니다. Stwo에서는 다음과 같은 수학적 정의를 따릅니다:

```
f(P) = Σ(i=0 to N-1) α^(N-1-i) * u_i(P)
```

여기서:

- `f(P)` 는 최종 컴포지션 다항식
- `u_i(P)` 는 i번째 제약 조건 다항식
- `α` 는 보안 랜덤 계수 (Fiat-Shamir 변환을 통해 생성)
- `N` 은 총 제약 조건의 개수

6.1.2 제약 조건에서 컴포지션 다항식으로의 변환 과정

단계 1: 제약 조건 평가 (Constraint Evaluation)

각 컴포넌트는 `ComponentProver` 트레이트를 구현하여 제약 조건을 평가합니다:

```rust
pub trait ComponentProver<B: Backend>: Component {
/// 평가 도메인에서 컴포넌트의 제약 조건 몫(quotient)을 평가합니다.
fn evaluate_constraint_quotients_on_domain(
&self,
trace: &Trace<'_, B>,
evaluation_accumulator: &mut DomainEvaluationAccumulator<B>,
);
}
```

단계 2: 도메인 평가 누적 (Domain Evaluation Accumulation)

`DomainEvaluationAccumulator` 는 다양한 크기의 제약 조건들을 효율적으로 결합합니다:

```rust
pub struct DomainEvaluationAccumulator<B: Backend> {
random_coeff_powers: Vec<SecureField>, // α의 거듭제곱들
sub_accumulations: Vec<Option<SecureColumnByCoords<B>>>, // 크기별 누적값
}
```

핵심 알고리즘:

1. 각 제약 조건 `u_i(P)` 에 대해 해당하는 랜덤 계수 `α^(N-1-i)` 를 곱합니다
2. 같은 도메인 크기를 가진 제약 조건들을 그룹화하여 누적합니다
3. 서로 다른 크기의 도메인들을 보간(interpolation)을 통해 통합합니다

단계 3: 계층적 보간 (Hierarchical Interpolation)

```
pub fn finalize(self) -> SecureCirclePoly<B> {
let mut cur_poly: Option<SecureCirclePoly<B>> = None;
let twiddles = B::precompute_twiddles(
CanonicCoset::new(self.log_size()).circle_domain().half_coset,
);


for (log_size, values) in self.sub_accumulations.into_iter().enumerate().skip(1
let Some(mut values) = values else { continue; };
if let Some(prev_poly) = cur_poly {
// 이전 다항식을 현재 도메인에서 평가
let eval = prev_poly.evaluate_with_twiddles(
CanonicCoset::new(log_size as u32).circle_domain(),
&twiddles,
);
// 현재 값들과 누적
B::accumulate(&mut values, &eval);
}
// 현재 레벨의 다항식 생성
cur_poly = Some(values.interpolate_with_twiddles(&twiddles));
}
cur_poly.unwrap_or_default()
```

```
}
```

6.1.3 랜덤 계수를 사용한 선형 결합 방식과 보안성

랜덤 계수 생성

Stwo는 Fiat-Shamir 변환을 통해 보안 랜덤 계수를 생성합니다:

```
// 증명자 측에서 랜덤 계수 추출
let random_coeff = channel.draw_secure_felt();


// 랜덤 계수의 거듭제곱 생성
let random_coeff_powers = B::generate_secure_powers(random_coeff, total_columns
```

보안성 분석

1. Schwartz-Zippel 보조정리 기반 보안성:

- 각 제약 조건이 독립적으로 0이 아닌 다항식이라면, 랜덤 선형 결합도 높은 확률로 0이 아닙니다
- 보안 매개변수는 필드 크기에 의해 결정됩니다 (QM31의 경우 약 124비트 보안성)

2. 지식 추출 가능성:

- 컴포지션 다항식이 올바르게 구성되었다면, 각 개별 제약 조건도 만족됨을 보장합니다
- 랜덤 계수의 예측 불가능성으로 인해 증명자는 모든 제약 조건을 만족해야 합니다

3. 제로 지식 특성:

- 랜덤 계수는 공개되지만, 개별 제약 조건 값들은 숨겨집니다
- 컴포지션 다항식만으로는 원본 계산 과정을 역추적할 수 없습니다

6.1.4 병렬 처리를 통한 성능 최적화 기법

SIMD 최적화

Stwo는 AVX512 등의 SIMD 명령어를 활용하여 병렬 처리를 수행합니다:

```
// VeryPackedSecureColumnByCoords를 사용한 벡터화 처리
let col = unsafe { VeryPackedSecureColumnByCoords::transform_under_mut(accum.co


// 병렬 처리 범위 설정
let range = 0..(1 << (eval_domain.log_size() - LOG_N_LANES - LOG_N_VERY_PACKED_
```

최적화 기법:

1. **벡터화된 필드 연산:** 16개 또는 32개의 필드 원소를 동시에 처리
2. **메모리 접근 패턴 최적화:** 캐시 친화적인 순차 접근 패턴 사용
3. **브랜치 예측 최적화:** 조건문을 최소화하여 파이프라인 효율성 향상

Rayon을 활용한 멀티스레딩

```
#[cfg(feature = "parallel")]
let iter = range
.into_par_iter()
.step_by(CHUNK_SIZE)
.zip(col.chunks_mut(CHUNK_SIZE));


iter.for_each(|(chunk_idx, mut chunk)| {
```

```
// 각 청크를 독립적으로 병렬 처리
for idx_in_chunk in 0..CHUNK_SIZE {
let vec_row = chunk_idx * CHUNK_SIZE + idx_in_chunk;
// 제약 조건 평가 및 누적
let eval = SimdDomainEvaluator::new(/* ... */);
let row_res = self_eval.evaluate(eval).row_res;
// 결과 누적
chunk.set_packed(idx_in_chunk, chunk.packed_at(idx_in_chunk) + row_res * denom_i
}
});
```

병렬 처리 전략:

1. **작업 분할:** 평가 도메인을 청크 단위로 분할하여 독립적 처리
2. **메모리 지역성:** 각 스레드가 연속된 메모리 영역을 담당
3. **동기화 최소화:** 락 없는 병렬 처리로 오버헤드 감소

성능 측정 결과

실제 벤치마크에서 관찰된 성능 향상:

- **SIMD 최적화:** 기본 구현 대비 8-16배 성능 향상
- **멀티스레딩:** CPU 코어 수에 비례한 선형 확장성
- **메모리 최적화:** 캐시 미스율 50% 이상 감소

6.1.5 구체적인 구현 예제

Wide Fibonacci 예제를 통한 컴포지션 다항식 생성 과정:

```rust
impl<const N: usize> FrameworkEval for WideFibonacciEval<N> {
fn evaluate<E: EvalAtRow>(&self, mut eval: E) -> E {
let mut a = eval.next_trace_mask();
let mut b = eval.next_trace_mask();
// N개의 피보나치 제약 조건 생성
for _ in 2..N {
let c = eval.next_trace_mask();
// 제약 조건: c - (a² + b²) = 0
eval.add_constraint(c.clone() - (a.square() + b.square()));
a = b;
b = c;
}
eval
}
}
```

이 제약 조건들은 다음과 같이 컴포지션 다항식으로 변환됩니다:

1. **제약 조건 수집:** 각 행에서 `c_i - (a_i² + b_i²)` 형태의 제약 조건 생성
2. **몫 계산:** 제약 조건을 vanishing polynomial로 나누어 몫 계산
3. **랜덤 선형 결합:** `f(P) = Σ α^i * quotient_i(P)` 형태로 결합
4. **보간:** 평가 결과를 다항식 계수로 변환

6.1.6 메모리 효율성과 확장성

메모리 사용량 최적화

```
// 지연 평가를 통한 메모리 절약
let  trace: TreeVec<Vec<Cow<'_, CircleEvaluation<_, _, _>>>> = if  need_to_exten
// 필요한 경우에만 확장
component_polys.map_cols(|col| Cow::Owned(col.evaluate_with_twiddles(eval_domain
} else {
// 기존 평가 재사용
component_evals.clone().map_cols(|c| Cow::Borrowed(*c))
};
```

확장성 고려사항

1. **도메인 크기 적응:** 제약 조건의 차수에 따라 평가 도메인 자동 조정
2. **컴포넌트 모듈화:** 독립적인 컴포넌트들의 조합으로 복잡한 계산 표현
3. **점진적 처리:** 메모리 제약이 있는 환경에서도 대규모 증명 생성 가능

이러한 설계를 통해 Stwo는 효율적이고 확장 가능한 컴포지션 다항식 생성을 실현하며, Circle STARK의 핵심 보안성을 유지하면서도 실용적인 성능을 제공합니다.

6.

2 OODS (Out-Of-Domain Sampling) 구현 분석

6.2.1 OODS 기법의 수학적 원리와 보안성

OODS(Out-Of-Domain Sampling)는 Circle STARK에서 다항식의 무결성을 검증하는 핵심 기법입니다. 이 방법은 다항식이 약속된 도메인 밖의 임의 점에서 올바른 값을 가지는지 확인함으로써 전체

다항식의 정확성을 보장합니다.

수학적 기반

1. 기본 원리:

OODS는 다음과 같은 수학적 사실에 기반합니다:

- 차수 `d` 인 다항식 `f(x)` 가 있고, 임의의 점 `z` 에서 `f(z) = v` 라고 주장한다면
- 몫 다항식 `q(x) = (f(x) - L(x)) / V(x)` 가 다항식이 되어야 합니다
- 여기서 `L(x)` 는 `L(z) = v`, `L(z*) = v*` 를 만족하는 선형 보간 다항식
- `V(x)` 는 `z` 와 `z*` 에서 소멸하는 vanishing polynomial

2. Circle 도메인에서의 적용:

Circle STARK에서는 복소수 켤레 대칭성을 활용합니다:

```
/// 복소수 켤레 선형 보간 함수
pub fn complex_conjugate_line(
point: CirclePoint<SecureField>,
value: SecureField,
p: CirclePoint<BaseField>,
) -> SecureField {
value + (value.complex_conjugate() - value) * (-point.y + p.y)
/ (point.complex_conjugate().y - point.y)
}
```

이 함수는 점 `(point, value)` 와 그 복소수 켤레 `(point*, value*)` 를 지나는 직선을 정의합니다.

보안성 분석

1. Soundness (건전성):

- 잘못된 다항식이 OODS 검사를 통과할 확률은 `d/|F|` 로 제한됩니다
- QM31 필드에서 차수 `2^20` 인 다항식의 경우, 실패 확률은 약 `2^(-104)` 입니다
- 이는 암호학적으로 무시할 수 있는 수준입니다

2. Zero-Knowledge (영지식성):

- OODS 점은 Fiat-Shamir 변환을 통해 무작위로 선택됩니다
- 증명자는 OODS 점을 미리 알 수 없으므로 특정 점에서만 작동하는 가짜 증명을 만들 수 없습니다
- 샘플링된 값들만으로는 원본 다항식을 복원할 수 없습니다

3. Knowledge Extraction (지식 추출):

- 유효한 OODS 증명을 생성할 수 있다면, 증명자는 실제로 올바른 다항식을 알고 있어야 합니다
- 이는 Circle STARK의 지식 건전성을 보장하는 핵심 요소입니다

6.2.2 Circle 도메인에서의 OODS 구현 방식

단계 1: OODS 점 선택

```
// 증명자와 검증자가 공통으로 사용하는 OODS 점 생성
let oods_point = CirclePoint::<SecureField>::get_random_point(channel);
```

이 점은 Fiat-Shamir 변환을 통해 결정론적으로 생성되지만 예측 불가능합니다.

단계 2: 마스크 포인트 수집

```
// 각 컴포넌트의 마스크 포인트들을 OODS 점 기준으로 수집
let mut sample_points = component_provers.components().mask_points(oods_point)
```

```
// 컴포지션 다항식의 마스크 포인트 추가
sample_points.push(vec![vec![oods_point]; SECURE_EXTENSION_DEGREE]);
```

마스크 포인트는 각 제약 조건이 참조하는 상대적 위치들을 나타냅니다.

단계 3: 다항식 평가

```
// 모든 다항식을 샘플 포인트에서 평가
let samples = self.polynomials()
.zip_cols(&sampled_points)
.map_cols(|(poly, points)| {
points.iter().map(|&point| PointSample {
point,
value: poly.eval_at_point(point),
}).collect_vec()
});
```

단계 4: OODS 몫 다항식 계산

```
pub fn compute_fri_quotients<B: QuotientOps>(
columns: &[&CircleEvaluation<B, BaseField, BitReversedOrder>],
samples: &[Vec<PointSample>],
random_coeff: SecureField,
log_blowup_factor: u32,
```

```
) -> Vec<SecureEvaluation<B, BitReversedOrder>> {

// 도메인 크기별로 그룹화

zip(columns, samples)

.sorted_by_key(|(c, _)| Reverse(c.domain.log_size()))

.group_by(|(c, _)| c.domain.log_size())

.into_iter()

.map(|(log_size, tuples)| {

let (columns, samples): (Vec<_>, Vec<_>) = tuples.unzip();

let  domain = CanonicCoset::new(log_size).circle_domain();

let  sample_batches = ColumnSampleBatch::new_vec(&samples);

// 몫 다항식 누적 계산

B::accumulate_quotients(

domain,

&columns,

random_coeff,

&sample_batches,

log_blowup_factor,

)

})

.collect()

}
```

6.2.3 샘플링 포인트 선택과 증명 검증 과정

샘플링 포인트 선택 전략

1. 배치 최적화 (Batch Optimization):

```
pub struct ColumnSampleBatch {
pub point: CirclePoint<SecureField>,
pub columns_and_values: Vec<(usize, SecureField)>,
}

impl ColumnSampleBatch {
pub fn new_vec(samples: &[&Vec<PointSample>]) -> Vec<Self> {
// 같은 점에서 샘플링되는 컬럼들을 그룹화
let mut grouped_samples = IndexMap::default();
for (column_index, samples) in samples.iter().enumerate() {
for sample in samples.iter() {
grouped_samples
.entry(sample.point)
.or_insert_with(Vec::new)
.push((column_index, sample.value));
}
}
// 안정적인 순서로 배치 생성
grouped_samples.into_iter()
.map(|(point, columns_and_values)| ColumnSampleBatch {
```

```
point, columns_and_values
})
.collect()
}
}
```

이 배치 최적화는 같은 점에서 여러 컬럼을 샘플링할 때 계산 효율성을 높입니다.

2. 복소수 켤레 대칭성 활용:

```
pub fn complex_conjugate_line_coeffs(
sample: &PointSample,
alpha: SecureField,
) -> (SecureField, SecureField, SecureField) {
// 직선 방정식 a*x + b - c*y = 0의 계수 계산
// (sample.y, sample.value)와 (conj(sample.y), conj(sample.value))를 지나는 직선
let a = sample.value.complex_conjugate() - sample.value;
let c = sample.point.complex_conjugate().y - sample.point.y;
let b = sample.value * c - a * sample.point.y;
(alpha * a, alpha * b, alpha * c)
}
```

증명 검증 과정

1. 몫 다항식 검증:

```
pub fn accumulate_row_quotients(
sample_batches: &[ColumnSampleBatch],
queried_values_at_row: &[BaseField],
quotient_constants: &QuotientConstants,
domain_point: CirclePoint<BaseField>,
) -> SecureField {
let denominator_inverses = denominator_inverses(sample_batches, domain_point);
let mut row_accumulator = SecureField::zero();
for (sample_batch, line_coeffs, batch_coeff, denominator_inverse) in izip!(
sample_batches,
&quotient_constants.line_coeffs,
&quotient_constants.batch_random_coeffs,
denominator_inverses
) {
let mut numerator = SecureField::zero();
for ((column_index, _), (a, b, c)) in zip_eq(&sample_batch.columns_and_values,
let value = queried_values_at_row[*column_index] * *c;
// 직선 방정식을 도메인 포인트에서 평가
let linear_term = *a * domain_point.y + *b;
numerator += value - linear_term;
}
// 배치별 결과를 랜덤 계수로 결합
```

```
row_accumulator = row_accumulator * *batch_coeff + numerator.mul_cm31(denominato
}
row_accumulator
}
```

2. 분모 역원 계산:

```
fn denominator_inverses(
sample_batches: &[ColumnSampleBatch],
domain_point: CirclePoint<M31>,
) -> Vec<CM31> {
let mut denominators = Vec::new();
// P가 Pr + uPi와 그 켤레 Pr - uPi를 지나는 직선 위에 있는지 확인
// (Pr - P).x * Pi.y - (Pr - P).y * Pi.x = 0 이어야 함
for sample_batch in sample_batches {
let prx = sample_batch.point.x.0;
let pry = sample_batch.point.y.0;
let pix = sample_batch.point.x.1;
let piy = sample_batch.point.y.1;
denominators.push((prx - domain_point.x) * piy - (pry - domain_point.y) * pix);
}
// 배치 역원 계산으로 효율성 향상
CM31::batch_inverse(&denominators)
```

```
}
```

6.2.4 성능 최적화 및 확장성

배치 처리 최적화

1. 메모리 지역성:

- 같은 점에서의 다중 샘플링을 배치로 처리하여 캐시 효율성 향상
- 연속된 메모리 접근 패턴으로 메모리 대역폭 최적화

2. 계산 재사용:

```
pub struct QuotientConstants {
pub line_coeffs: Vec<Vec<(SecureField, SecureField, SecureField)>>,
pub batch_random_coeffs: Vec<SecureField>,
}
```

```
pub fn quotient_constants(
sample_batches: &[ColumnSampleBatch],
random_coeff: SecureField,
) -> QuotientConstants {
QuotientConstants {
line_coeffs: column_line_coeffs(sample_batches, random_coeff),
batch_random_coeffs: batch_random_coeffs(sample_batches, random_coeff),
}
```

```
}
```

미리 계산된 상수들을 재사용하여 중복 계산을 방지합니다.

FRI와의 통합

1. 몫 다항식의 FRI 약속:

```
// OODS 몫 다항식들을 FRI 프로토콜에 제출
let fri_prover = FriProver::<B, MC>::commit(
channel,
self.config.fri_config,
&quotients,
self.twiddles
);
```

2. 쿼리 일관성 검증:

```
// FRI 쿼리 위치에서 몫 다항식 값들이 일치하는지 확인
let quotient_evals_at_queries = fri_answers_for_log_size(
log_size,
&samples,
random_coeff,
query_positions,
&mut queried_values,
n_columns,
```

```
)?;
```

6.2.5 보안성 분석 및 공격 저항성

알려진 공격 방법과 대응

1. 선택적 실패 공격 (Selective Failure Attack):

- **공격:** 특정 OODS 점에서만 실패하는 악의적 증명 시도
- **대응:** Fiat-Shamir 변환으로 예측 불가능한 OODS 점 생성

2. 저차수 테스트 우회 (Low-Degree Test Bypass):

- **공격:** 높은 차수의 다항식을 낮은 차수로 위장
- **대응:** FRI 프로토콜과의 통합으로 차수 검증 강화

3. 일관성 공격 (Consistency Attack):

- **공격:** 서로 다른 약속 간의 불일치 악용
- **대응:** 컴포지션 다항식과 트레이스 값의 교차 검증

실제 보안 매개변수

Stwo에서 사용되는 보안 매개변수들:

- **필드 크기:** QM31 (약 124비트 보안성)
- **FRI 폴딩 인수:** 일반적으로 4 (2비트씩 폴딩)
- **쿼리 수:** 보안 수준에 따라 조정 (일반적으로 80-100개)
- **작업 증명:** 추가적인 보안성 제공

이러한 OODS 구현을 통해 Stwo는 효율적이면서도 암호학적으로 안전한 다항식 약속 검증을 실현하며, Circle STARK 프로토콜의 핵심 보안성을 보장합니다.